# Machine Learning — Project 3

scikit-learn — Program 1 and Program 2

This report matches the course rubric: what each program implements and how to read the results. Code: `program1.ipynb`, `program2.ipynb`. Figures load from `figures/` (`random_state=42` throughout).

## 1. Program 1 — Classification (`make_moons`)

### 1.1. Rubric alignment

1. **Data (1 pt):** `make_moons(n_samples=10000, noise=0.4)`; `train_test_split` train/test split (here: 8000 / 2000 samples).
2. **Training and errors (2 pts):** `LogisticRegression`, `SVC` (SVM), `RandomForestClassifier`, `VotingClassifier` combining all three with `soft` voting; train and test **error rate** as `1 - accuracy_score`.
3. **Figure and analysis (2 pts):** Below: class boundaries for the **VotingClassifier** only (`contourf` + test points).

### 1.2. Result table

Values are reproducible with the same `random_state` as in the notebook.

| Model | Train error | Test error |
|---|---|---|
| LogisticRegression | 0.1703 | 0.1585 |
| SVM (RBF) | 0.1362 | 0.1260 |
| RandomForest | 0.0000 | 0.1495 |
| VotingClassifier (soft) | 0.1006 | 0.1295 |

### 1.3. Short discussion

`make_moons` is not linearly separable, so logistic regression tends to be weakest on the test split. SVM (RBF) draws a non-linear boundary and generalizes better. Random forest drives training error almost to zero (strong capacity / overfitting tendency); test error rises again because of label noise. **VotingClassifier** blends base learners and can trade off variance and bias; here test error is close to SVM and better than logistic regression alone.

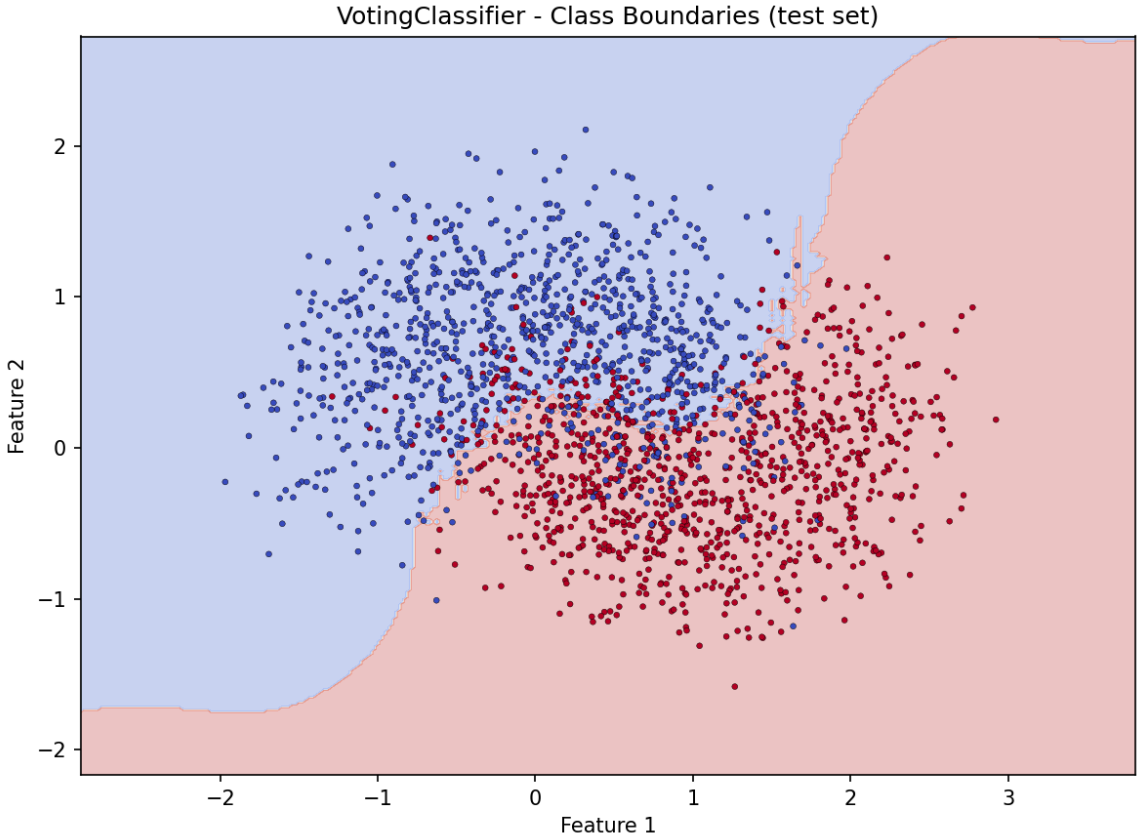


Figure 1: **Program 1 — Figure 1.** `VotingClassifier` decision regions; points are the test set, colors are true labels.

## 2. Program 2 — Regression (`dane*.txt`)

### 2.1. Rubric alignment

1. **Dataset handling (2 pts):** Two files (`dane1.txt`, `dane2.txt`); each line `input output`; `train_test_split`.
2. **Models and verification (3 pts):** `LinearRegression`; polynomial model via `PolynomialFeatures` + `LinearRegression` in a `make_pipeline`; comparison; `r2_score` on the test set.

### 2.2. Test $R^2$ table (matches notebook)

| Dataset | Linear (test) | Poly degree 3 (test) |
|---|---|---|
| dane1 | −0.1475 | 0.9915 |
| dane2 | 0.8752 | 0.9999 |

### 2.3. Notes on the numbers

These values were recomputed with the same loader, split (`test_size=0.2`, `random_state=42`), and models as in `program2.ipynb`. **dane2** polynomial $R^2$ is **not** exactly 1.0: it is about **0.999895**. Rounding to three decimals would show `1.000`, which can look like a "perfect" fit and is misleading.

Both files are small tabular datasets (dane1: 41 samples, dane2: 61 samples), so the test fold has only a dozen points. With a cubic polynomial, the model is very flexible relative to that sample size, so an $R^2$ extremely close to 1 on the test split is plausible even when the underlying relationship is smooth—it does not prove zero error in general, only strong agreement on this particular split.

### 2.4. Short discussion

For **dane1**, linear test $R^2$ is negative: on the test fold the linear model is worse than predicting the mean baseline. The relationship is clearly curved, so degree-3 polynomial regression fits much better. For **dane2**, the linear model already explains most variance; the polynomial adds a smaller but measurable gain. The notebook plot overlays data, linear fit, and cubic fit with test $R^2$ in the legend.

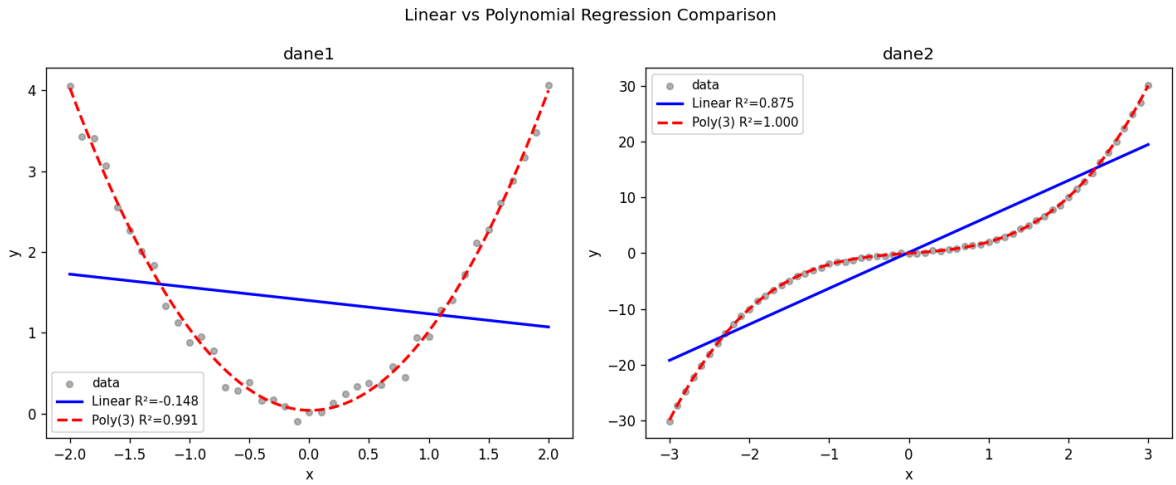


Figure 2: **Program 2 — Figure 2.** Both datasets: scatter, linear and degree-3 polynomial curves, test $R^2$ in the legend (notebook export).

## 3. Conclusion

Both programs satisfy the rubric: Program 1 covers data, four models plus voting, error reporting, and the voting decision-region figure; Program 2 covers two `dane` files, linear vs polynomial models, test $R^2$ comparison, and a comparative figure—with table values aligned to the notebook and rounding caveats stated explicitly.